

Fault-Tolerance

Definition

Fault-tolerance is the ability of a system to **continue functioning even when some components fail**, while preventing permanent damage such as:

- Data loss
- Full system crash
- Service outage
- Corrupted transactions

It ensures the system degrades gracefully instead of failing catastrophically.

Simple Meaning

If part of the system breaks, the whole system should still work (fully or partially).

Fault-Tolerance vs Availability

Concept	Meaning
Availability	Percentage of time system is up and serving requests
Fault-Tolerance	Ability to survive failures and continue operating

Example

- Website is up 99.99% time = Availability
- Database node crashes but app still works = Fault-Tolerance

Why Fault-Tolerance Matters

Without fault-tolerance:

- Single server crash can stop business
- Data may be lost
- Users face downtime
- Revenue impact
- Poor user trust

With fault-tolerance:

- Users may not notice failures
- Engineers get time to fix issues
- Safer operations at scale

Goals of Fault-Tolerance

1. Continue Service During Failure

Some parts fail, core service still runs.

2. Graceful Degradation

Reduced functionality instead of total outage.

Example:

- Search works, recommendations disabled.

3. Prevent Permanent Damage

Avoid:

- Data corruption
- Duplicate payments
- Lost orders

4. Fast Recovery

System restores quickly.

Common Faults in Distributed Systems

- Server crash
- Database failure
- Disk failure
- Network partition
- High latency
- Memory leak
- Third-party API outage
- DNS failure
- Deployment bug
- Human error

Fault-Tolerance Techniques

1. Redundancy

Run multiple instances.

Example:

- 3 app servers
- If 1 fails, others serve traffic.

2. Replication

Keep multiple copies of data.

Examples:

- Primary + replicas
- Multi-region replication

Prevents data loss.

3. Failover

Automatically switch to healthy component.

Example:

Primary DB fails → Replica promoted.

4. Load Balancer Health Checks

Load balancer removes unhealthy nodes.

5. Retry Mechanism

Temporary failures can be retried.

Use:

- Exponential backoff
- Limited retries

6. Circuit Breaker Pattern

If downstream service failing repeatedly:

- Stop sending requests temporarily
- Prevent cascading failures

Example:

Hystrix

7. Timeout Control

Never wait forever.

Example:

- Payment API timeout after 2 sec.

8. Queue-Based Buffering

Use queues during spikes or failures.

Examples:

Apache Kafka

RabbitMQ

9. Auto Healing / Self Healing

Automatically replace failed instances.

Example:

Kubernetes restarts crashed pods.

10. Backups + Disaster Recovery

Recover data after severe failures.

Graceful Degradation Examples

Netflix

If recommendation engine fails:

- Video streaming still works.

Amazon

If review service fails:

- Product purchase still available.

Failure Design

Design how system behaves during failures.

Ask:

- What if DB is slow?
- What if cache unavailable?
- What if payment gateway down?
- What if API returns wrong data?
- What if network drops?

Silent Failures

Most dangerous failures.

System returns 200 OK but data wrong.

Examples:

- Payment deducted but order not created
- Cache stale data served
- Metrics pipeline broken silently

Need:

- Audits
- Monitoring
- Reconciliation jobs

Interview Discussion Points

When asked fault-tolerance, mention:

1. Remove single points of failure
2. Replication
3. Retries + timeout
4. Circuit breaker
5. Queue buffering
6. Monitoring + alerts
7. Auto failover
8. Disaster recovery

Real Example: Payment System

If payment provider unavailable:

- Queue request
- Retry later
- Show pending status
- Notify user later

Instead of crashing checkout.

One-Line Memory Trick

Fault-tolerance = Failures happen, but users should barely notice.